SRI RAMACHANDRA
INSTITUTE OF HIGHER EDUCATION AND RESEARCH
(Category - I Deemed to be University) Porur, Chennai
SRI RAMACHANDRA FACULTY OF ENGINEERING AND TECHNOLOGY

CrickSense

Ball-by-Ball Analysis of a Cricket Broadcast from a Single Camera Feed

The Complete Project Guide

Everything about the project - the cricket, the computer vision, every module, the results, the code and how to run it - explained from the beginning

KABILASH S (E0323036) PAVITRA P (E0323038)

Guide: Dr. Nirmala B, Department of Artificial Intelligence and Data Analytics

Sri Ramachandra Faculty of Engineering and Technology, SRIHER, Chennai

October 2026

Contents

1. What CrickSense Is 1
- 1.1 What Goes In and What Comes Out 1
- 1.2 The Results in One Table 1
- 1.3 Three Ideas That Run Through the Whole Project 2

2. Cricket Basics for This Project 3
- 2.1 The Pitch and Its Painted Lines 3
- 2.2 Balls, Overs and the Score 3
- 2.3 Length and Line 3
- 2.4 What a Broadcast Shows 4

3. Computer Vision and AI Concepts Used 5
- 3.1 Images, Pixels and Video 5
- 3.2 Homography: From Pixels to Metres 5
- 3.3 Following a Moving Camera: Features, RANSAC and Optical Flow 5
- 3.4 Object Detection with YOLO 6
- 3.5 Tracking and Assignment 6
- 3.6 Pose Estimation 6
- 3.7 Reading Text: OCR and Templates 7
- 3.8 Language Models, Speech and Source Separation 7

4. How the System Is Organised 8
- 4.1 The Big Picture 8
- 4.2 The Ten Passes 9
- 4.3 Why Passes Joined by Files? 9
- 4.4 One Command 9

5. Step 1: Calibration 10
- 5.1 Finding the Pitch Automatically 10
- 5.2 Following the Camera Through Every Frame 11
- 5.3 How Well It Works 11

6. Step 2: Reading the Score Bar 12
- 6.1 How It Is Read 12
- 6.2 Trusting a Reading 12

7. Step 3: Players 14

8. Step 4: The Ball 16
- 8.1 Finding the Delivery 16
- 8.2 The Bounce: a Clever Trick 16
- 8.3 Speed 17

9. Step 5: Events and the Score Engine 19
- 9.1 What Each Ball Produced 19
- 9.2 The Score Engine 19

10. Step 6: Batter Pose 20

11. Steps 7-9: Commentary, Voice and the Overlay 21
- 11.1 Commentary That Cannot Make Things Up 21
- 11.2 Voice 22
- 11.3 The Overlay and the Crowd Sound 22

12. Live Mode 24
- 12.1 Workers That Never Fall Behind 24
- 12.2 Making It Fast Enough 24
- 12.3 The Delivery Card 24

13. Following One Ball Through the System....26
14. Testing and Measurement....28
14.1 How Results Were Measured....28
14.2 All Results....28
14.3 Processing Time....29
15. What Did Not Work, and Why....30
16. The Code, Files and Configuration....31
16.1 Where Everything Lives....31
16.2 Configuration Files....31
16.3 The Output Folder of a Clip....32
17. Running the Project....33
17.1 Setting Up....33
17.2 Everyday Commands....33
17.3 A New Match....33
18. How the Project Developed....34
19. Limitations and Future Work....35
19.1 Limitations....35
19.2 Future Work....35
20. Glossary....36
References....37

CHAPTER 1

What CrickSense Is

CrickSense is a computer-vision system that **watches a cricket match on television and measures it, ball by ball**. It needs only the broadcast video - the same picture any viewer sees - and nothing else: no sensors in the ground, no special cameras, and no data from the broadcaster. From that video it works out who bowled each ball and who faced it, where the ball bounced, how fast it was, what the batter did, and what the ball produced (runs, a boundary, a wicket or an extra). It keeps the score by the Laws of Cricket, writes and speaks commentary about each ball, and draws everything on top of the video.

This guide explains the whole project so that a reader who starts with no special knowledge can understand every part of it: the cricket ideas it relies on (Chapter 2), the computer-vision and AI ideas (Chapter 3), how the system is organised (Chapter 4), each step in detail (Chapters 5 to 12), one ball followed through the whole system (Chapter 13), how the results were measured (Chapter 14), what was tried and dropped (Chapter 15), the code and how to run it (Chapters 16 and 17), and how the project developed (Chapter 18).

1.1 What Goes In and What Comes Out

Table 1.1: Inputs and outputs

Goes in	Comes out
A cricket broadcast video (MP4, 1080p, 25 frames per second): a short clip, a whole match, or a live stream	An **overlay video**: the broadcast with named players, the ball's path and bounce, a card per ball, banners and spoken commentary
A **team sheet** for the match (the two squads and wicketkeepers), a small text file	A **pitch map** (where every ball bounced) and a **wagon wheel** (where the runs went)
Optional: an API key for the commentary language model (free tier)	**JSON data files** for every step: the camera, the score bar, players, the ball, events, the match state, commentary
	A **live window** that shows names, the score and each ball's result as the match plays

1.2 The Results in One Table

Every result was measured on video that was not used to build the part being measured. Table 1.2 gives the headline numbers; Chapter 14 explains each.

Table 1.2: Headline results

What was measured	Result
Pitch calibration against points clicked by hand	7 of 8 crease corners within 3.3 pixels
Calibration on 20 random clips, no clicks	14 calibrated, none wrongly
Score bar over two whole matches	all 4 final scores read exactly
Deliveries and outcomes on three hand-checked clips	21 of 21 deliveries found; 21 of 21 outcomes right
Score engine against the broadcaster's score	230 of 230 checks agree
Our speed estimate against the broadcaster's radar	5.7% median error on the final
Live mode	each ball's result about 0.5 s after the score bar; radar speed in about 0.1 s

1.3 Three Ideas That Run Through the Whole Project

- **Measure, or say it is not measured.** Anything the broadcast does not show - runs taken by running, wides, no-balls - is read from the broadcaster's score bar and labelled so. Our own speed estimate is always shown with "≈" and the word "estimate".
- **Precision first.** A wrong measurement is worse than a missing one, because a missing one is visible and a wrong one silently corrupts everything after it. So every step rejects doubtful results rather than guessing.
- **Prove it with numbers.** Every design decision was made by measuring alternatives on real broadcast footage. The project keeps a decisions log (`docs/decisions.md`) recording each measurement and what it led to.

CHAPTER 2

Cricket Basics for This Project

This chapter explains just enough cricket to follow the rest of the guide.

2.1 The Pitch and Its Painted Lines

Cricket is played on an oval field. In the middle is the **pitch**, a strip of hard, pale-brown ground **20.12 metres** long between two sets of **stumps** (three wooden posts). White lines called **creases** are painted at each end, and their positions are fixed by the Laws of Cricket:

- the **bowling crease** runs through the stumps;
- the **popping crease** is **1.22 m** in front of the stumps - the batter is safe behind it;
- the **return creases** run at right angles, **1.32 m** either side of the middle stump.

Because these lines are the same on every cricket pitch in the world, a computer can recognise them and use them as a ruler. This is the key that lets CrickSense measure distances in metres (Chapter 5).

> **In CrickSense's coordinates**
> The pitch is measured in metres: **x** runs along the pitch from the striker's stumps (x = 0) towards the bowler's stumps (x = 20.12); **y** runs across it (0 on the middle-stump line, positive to the right as seen on screen).

2.2 Balls, Overs and the Score

- A **bowler** delivers the ball from one end to the **striker** (the batter facing) at the other end. The other batter, the **non-striker**, stands at the bowler's end.
- Six legal balls make an **over**. Overs are written like **12.3**: 12 overs completed, 3rd ball of the 13th. A T20 innings has at most 20 overs.
- The batters **run** between the ends to score; hitting the ball over the boundary on the bounce scores a **four**, on the full a **six**. When the batters run an odd number of runs, they swap ends, so the other batter faces next (the **strike** changes). The strike also changes at the end of each over.
- Extras: a **wide** (the ball too far from the batter) and a **no-ball** (an illegal delivery, e.g. the bowler's front foot over the line) give a run and do not count as one of the over's six balls.
- A **wicket** is when a batter is out (bowled, caught and so on); the next batter comes in. The score is written **runs-wickets**, e.g. 93-3.

2.3 Length and Line

Where the ball bounces matters most to coaches and commentators. **Length** is how far from the striker's stumps the ball bounces; **line** is how far to the side. CrickSense uses these length zones for pace bowling:

Table 2.1: Length zones

Zone	Bounce distance from the striker's stumps	What it means
Yorker	less than 2 m	lands at the batter's feet - hard to hit
Full	2 to 6 m	pitched up, invites a drive
Good	6 to 8 m	the batter is unsure whether to go forward or back
Short	more than 8 m	bounces high towards the batter's body

2.4 What a Broadcast Shows

A TV broadcast of cricket has a **main camera** standing far behind the bowler, looking down the pitch with a long zoom lens. For each delivery the picture shows the bowler running in and bowling, then the camera zooms and pans to follow the ball and the batter. Within a second or two of the shot, the director cuts to other cameras: the fielder chasing the ball, close-ups, replays from other angles, the crowd. At the bottom of the screen a **score bar** shows the batting team's score, the overs, the two batters (a bat icon marks the striker) with their runs and balls, the bowler and their figures, and often the **radar speed** of the last ball (Figure 6.1 in Chapter 6).

CHAPTER 3

Computer Vision and AI Concepts Used

This chapter explains each technique in plain terms, so that the later chapters can simply refer to it.

3.1 Images, Pixels and Video

A digital image is a grid of **pixels**; each pixel has a colour stored as three numbers (blue, green, red in OpenCV). A 1080p frame has 1920 × 1080 pixels. A pixel position is written (u, v): u from the left, v from the top. A video is a sequence of frames; at **25 frames per second (fps)** one frame lasts 0.04 s. A fast ball travels 3-6 m between two frames, which is why it often appears as a blurred streak only a few pixels wide.

3.2 Homography: From Pixels to Metres

The pitch is flat. When a camera looks at a flat surface, every point on the surface and its pixel in the image are related by a **homography** - a 3 × 3 matrix H. In **homogeneous coordinates** (a point written with an extra 1), one matrix multiplication converts a pixel into pitch metres:

```
[x']       [u]              x = x' / w'
[y'] = H · [v]     and       y = y' / w'
[w']       [1]
```

H has 8 free numbers, so **four** known point pairs fix it - for example the four corners where the popping creases meet the return creases, whose pitch positions are known from the Laws. Fitting H to points is done with the standard **Direct Linear Transform** [6]. Once H is known for a frame, any pixel on the ground - a player's feet, the ball at the moment it bounces - can be converted to metres on the pitch. This is called **camera calibration** for the ground plane.

> **Why one homography is not enough**
>
> The broadcast camera pans and zooms, so H changes every frame. CrickSense calibrates a few **reference frames** and then follows the camera from frame to frame (Section 3.3).
>
> A homography only describes the ground. For points above the ground (the ball in flight) the full 3D camera is recovered from H, assuming square pixels and the image centre as the principal point (`calibration/camera_pose.py`).

3.3 Following a Moving Camera: Features, RANSAC and Optical Flow

To know H for every frame, CrickSense finds how each frame relates to a calibrated reference frame:

- **ORB features** [14]: small distinctive patches (corners, texture) found in both images, each with a binary description so the same patch can be matched between images.
- **Matching and the ratio test**: each feature is matched to its most similar feature in the other image; a match is kept only if it is clearly better than the second-best.

- **RANSAC** [15] (Random Sample Consensus): some matches are always wrong. RANSAC repeatedly picks 4 matches at random, fits a homography, counts how many other matches agree with it (the **inliers**), and keeps the best. CrickSense requires at least 80 inliers spread over the frame.
- **Optical flow** [16], [17]: instead of matching against a reference, track a few hundred corners from the previous frame to the current one with the pyramidal **Lucas-Kanade** method. It is about 10 times faster (about 10 ms a frame) and is used by the live mode, with a full reference match every few frames to stop small errors from adding up.
- **Smoothing**: per-frame estimates jitter by a few pixels; a **Savitzky-Golay** filter [21] smooths them over 11 frames, after replacing obvious outliers.

3.4 Object Detection with YOLO

An **object detector** is a neural network that takes an image and returns boxes around objects, each with a class (person, ball) and a **confidence** between 0 and 1. **YOLO** ("You Only Look Once") [10] does this in a single pass of a convolutional network, fast enough for video. CrickSense uses the Ultralytics implementations [11]: **YOLOv8m** trained on the COCO dataset [23] for people, two **YOLOv8l** models for the cricket ball (one trained by the team, one **fine-tuned** - further trained - on 94 hand-checked frames of this broadcast), a YOLOv8 bat model, and **YOLO11x-pose** for body keypoints. The ball models run at the full 1920-pixel width, because at 1280 pixels the 6-15 pixel ball is mostly lost.

> **FP16 (half precision)**
> Neural networks normally compute with 32-bit numbers (FP32). On a modern GPU, 16-bit numbers (FP16) are much faster with almost the same results. In the live mode the ball model went from 168 ms to 51 ms per frame, with exactly the same detections on 48 of 48 test frames.

3.5 Tracking and Assignment

- **Multi-object tracking** links detections across frames so each person keeps an identity. **BoT-SORT** [13] predicts where each tracked person will be (a **Kalman filter**, which estimates position and speed from noisy measurements) and matches new boxes to those predictions. It also compensates for camera motion; CrickSense gives it the exact camera motion from the calibration instead of estimating it from the image.
- **The Hungarian algorithm** solves an **assignment problem**: given a score for every (player, role) pair, it finds the one-to-one assignment with the best total. CrickSense uses it so that two players can never both be the striker.

3.6 Pose Estimation

A **pose model** finds a person's body **keypoints** - 17 points such as wrists, elbows, hips, knees and ankles (the COCO-17 layout). From the keypoints CrickSense measures the striker's

footwork (where the feet moved on the pitch) and the swing (how fast the wrists moved compared with the hips).

3.7 Reading Text: OCR and Templates

Optical character recognition (OCR) turns pictures of text into characters. CrickSense uses **Tesseract** [18] for names. For the score digits, which Tesseract misread, it uses **template matching**: the score box has a fixed font on a flat colour, so each digit is cut out and compared with stored pictures (templates) of the ten digits in that font. The templates were **harvested automatically** from the broadcast itself, from 3,453 readings where Tesseract had been unambiguous.

3.8 Language Models, Speech and Source Separation

- A **large language model (LLM)** generates text. CrickSense sends it a short list of facts about a ball and asks for one line of commentary. LLMs can **hallucinate** - state things that are not in their input [19] - so every line is checked against the facts (Chapter 11).
- **Text-to-speech (TTS)** turns text into a voice. CrickSense uses **Kokoro-82M** [22], a small model that runs on the laptop with no internet or key.
- **Source separation** splits a mixture of sounds into its parts. **Demucs** [20] separates voices from everything else in a soundtrack; CrickSense uses it to remove the TV commentators while keeping the crowd.

CHAPTER 4

How the System Is Organised

4.1 The Big Picture

Figure 4.1 shows the whole system. It has three parts. The **offline pipeline** (top) analyses a video in ten steps called **passes**; each pass reads the files written by earlier passes and writes its own. The **live path** (middle) runs the time-critical parts on a video as it plays. The **verification** layer (bottom) checks results before they are shown.

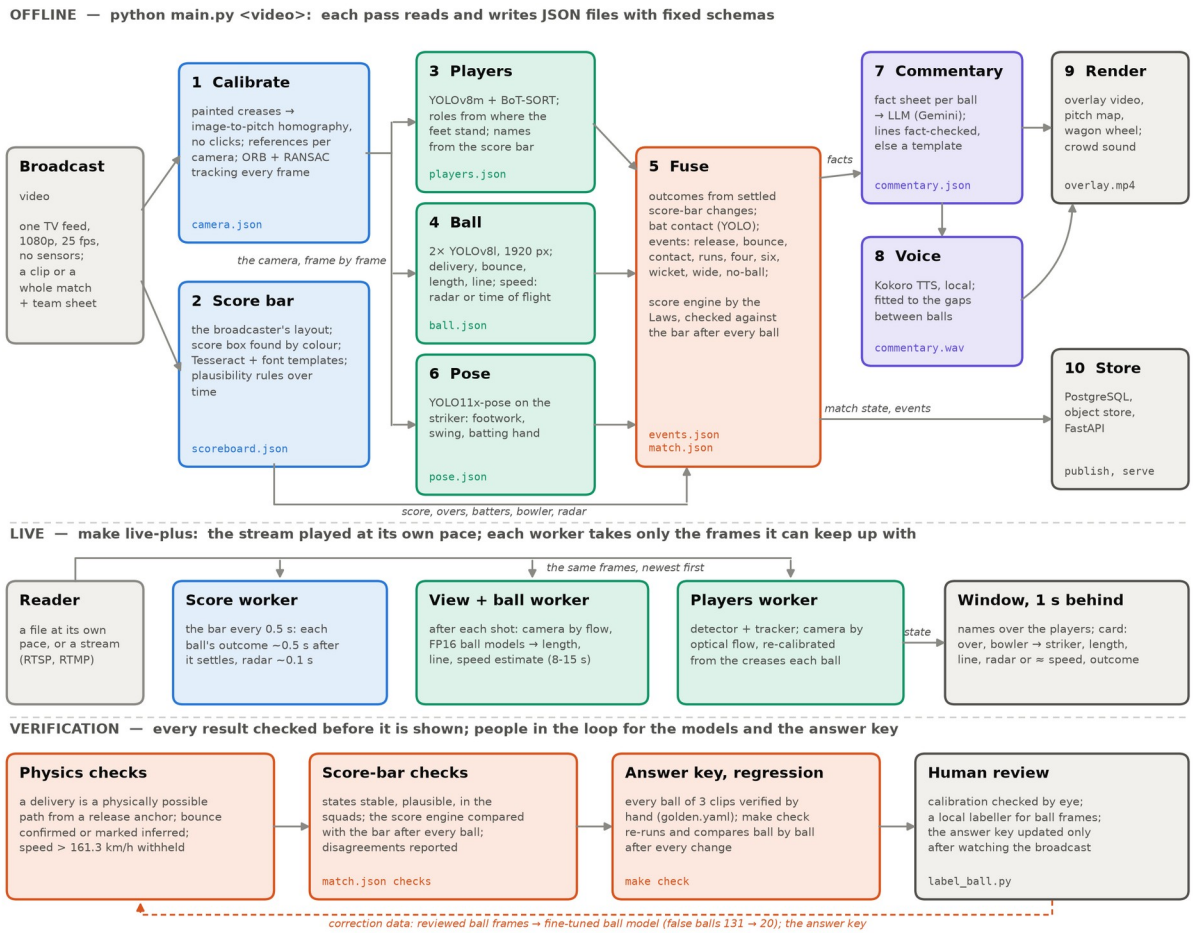


Figure 4.1: The CrickSense architecture: the offline passes and the files they exchange, the live path, and verification

4.2 The Ten Passes

Table 4.1: The passes, in order

#	Pass	What it does	Writes
1	calibrate	finds the pitch from its painted creases; the camera for every frame	camera.json
2	scoreboard	reads the score bar every 0.5 s	scoreboard.json
3	players	detects and tracks people; roles and names	players.json
4	ball	finds each delivery: release, track, bounce, length, line, speed	ball.json
5	fuse	events and the match state by the Laws, checked against the bar	events.json, match.json
6	pose	the striker's footwork, swing and batting hand	pose.json
7	commentary	fact sheets, LLM lines with fact check, templates	commentary.json
8	voice	speaks the lines and places them in time	commentary.wav, voice.json
9	render	overlay video, pitch map, wagon wheel; crowd sound	overlay.mp4, PNG images
10	publish / serve	database and HTTP API	PostgreSQL rows, API

4.3 Why Passes Joined by Files?

- **Each step can be checked alone.** If the score is wrong, you can open `scoreboard.json` and see whether the reading or the reasoning was at fault.
- **Each step can be re-run alone.** Changing the commentary does not mean re-running the 40-minute calibration.
- **Strict schemas.** Every file follows a fixed structure (defined with Pydantic in `src/cricsense/schema/models.py`), so a pass cannot silently pass on malformed data.
- **Finished steps are reused.** A step whose output already exists is skipped, so an interrupted run continues where it stopped.

4.4 One Command

All of it runs from one command: `python main.py <video>`. Given a clip, it runs every pass. Given a whole match, it first checks the team sheet, reads the score bar across the match, prints both final scores and lists every over; then `--overs 1:12 2:2` (innings:over), `big` or `all` cuts those overs out and analyses each one.

CHAPTER 5

Step 1: Calibration

Calibration answers the question: **for this frame, which pixel is which point on the pitch?** Everything that measures in metres depends on it - player positions, the bounce, length, line and speed.

5.1 Finding the Pitch Automatically

The earlier prototype needed a person to click the four crease corners. CrickSense finds them itself (`calibration/auto.py`):

- **White-line candidates.** A **top-hat filter** highlights thin bright lines; a line is kept only if there is pitch-coloured ground (pale brown) on both sides, which removes advertising boards, logos and players' clothes.
- **Corner hypotheses.** Pairs of near-horizontal lines (the creases across the pitch) and steep segments (the return creases) cross at candidate corners; each set of four gives a homography.
- **Scoring.** Each homography is used to draw the full regulation crease pattern back into the image. Its score is the fraction of that pattern that lands on white paint, plus whether the stumps stand where the middle-stump base projects. A score of 0.72 or more is acceptable, 0.78 or more strong.
- **Rejecting impossible cameras.** A 3D camera is recovered from each homography; one that would need, for example, a camera below the ground is rejected.
- **Agreement.** Frames are grouped by where their calibration places the camera. A camera view is accepted only with **two strong frames that agree**, or three, because a batter standing on a crease can fool a single frame.

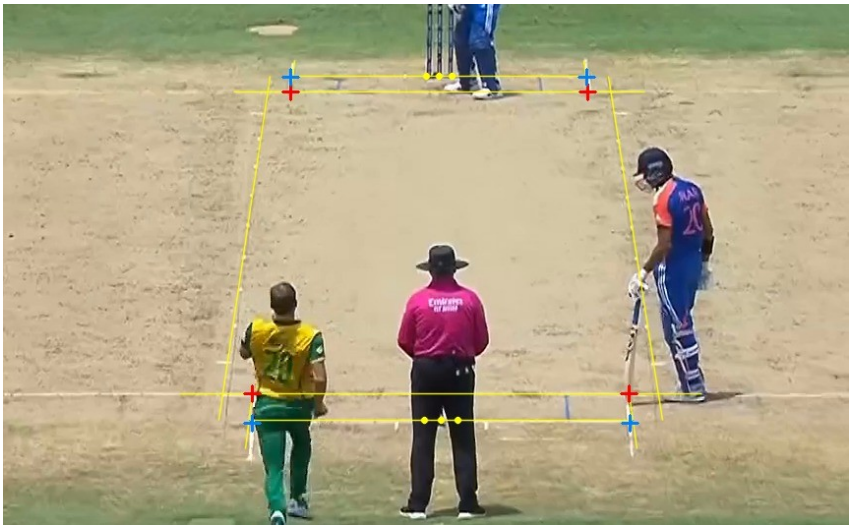


Figure 5.1: A calibrated frame: the regulation creases drawn through the fitted homography (yellow) lie on the painted creases

5.2 Following the Camera Through Every Frame

Each camera keeps up to **six reference frames**, at least 10 seconds apart, because the camera zooms so much during a delivery that one reference stops matching. Every frame is matched with ORB and RANSAC to a reference (Section 3.3); its homography is the reference's calibration combined with the frame-to-reference homography. Frames with almost no pitch in view (crowd, close-ups, graphics) are skipped, and the score bar and logo are masked out so that matching on them does not drag the estimate towards "no motion". Within a shot, a frame is first matched only against the reference the previous frame used, which made tracking 2,693 → 446 s on Bumrah's over with identical results.

5.3 How Well It Works

- On the development frame, **7 of 8 corners were within 3.3 pixels** of corners clicked by hand; where the hand calibration had drifted, the automatic one sat on the paint.
- On **20 random clips** of the final, 14 calibrated and **none was calibrated wrongly**. The misses were clips with one or no clear pitch view.
- Calibration is the slowest step: about 40 minutes for a 6-minute clip, because every frame is matched at full resolution. Half-resolution matching was tried and rejected: 1.9 times faster but the corner error grew from 1.8 to 3.1 pixels (median).

CHAPTER 6

Step 2: Reading the Score Bar

The broadcaster's score bar is the most reliable source of what happened on each ball. CrickSense reads it every 0.5 seconds (`identity/scoreboard_ocr.py`, `passes/scoreboard.py`).



Figure 6.1: The score bar's fields (blue boxes) and the striker's bat icon (orange). Read here: Axar 38 (26), Kohli* 41 (35), 93-3 after 12 overs, bowler Nortje 0-4

6.1 How It Is Read

- **A layout file** (`configs/scoreboard.yaml`) gives each field's rectangle in a 1920 × 1080 frame: the two batters' names, runs and balls, the score, the overs, the bowler, the bowler's figures, and the line that shows the radar speed. A different broadcaster only needs a new layout file, not new code.
- **Is the bar on screen?** The pink score box is checked by colour first; if it is missing (a replay or graphic covers it), nothing is read.
- **The striker** is the batter with the yellow bat icon beside the name.
- **Names** are read with Tesseract and snapped to the nearest name in the team sheet's squads, so an OCR slip like "KOHL1" still becomes KOHLI.
- **The score digits** are read with font templates (Section 3.7): leave-one-out test 250 of 250 correct.
- **When the bar moves** (it slides by about 20 px, and the chase shows a compact variant), the pink score box is found by colour anywhere in the band and its digits and the overs beside it are read from there.

6.2 Trusting a Reading

A single reading can be wrong - a graphic may sit on the bar for a moment. So a state is trusted only if it is **stable** (seen in two readings in a row), **complete**, made of **squad names**, never **goes backwards**, and is **reachable** from the previous state in the time that passed (at most one ball per 15 seconds and 8 runs per ball). The last rule mattered: a stable misread of "1.0" overs as "10.0" had once blocked every genuine state after it.

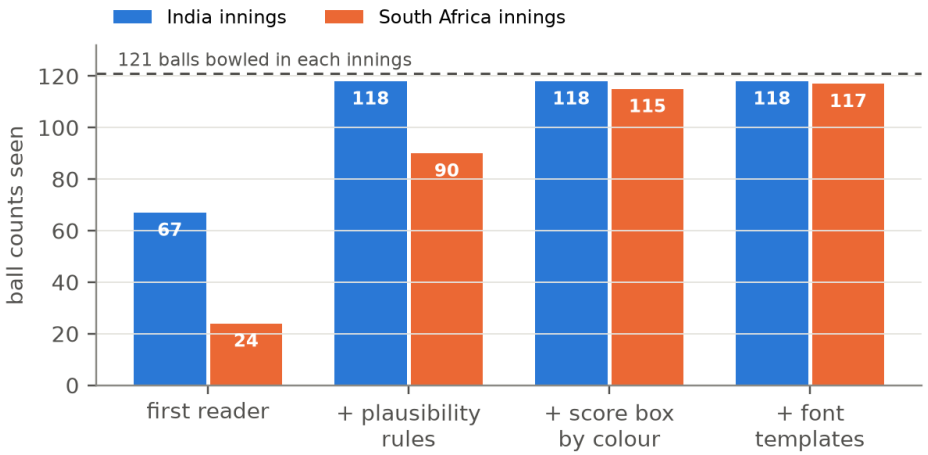


Figure 6.2: How each fix raised the ball counts read over the whole final (121 balls were bowled in each innings)

Over both whole matches, all four final scores were read exactly (176-7 and 169-8 in the final, 205-5 and 181-7 in the Super 8 match), and 117-121 of the 121 ball counts of each innings were seen.

CHAPTER 7

Step 3: Players

The players pass (`passes/players.py`) finds every person on the main camera's frames and works out who is who.

- **Detection.** The COCO-trained **YOLOv8m** finds people at 1280 pixels. (A five-class cricket model retrained in the earlier prototype scored 0.895 mAP50 on its own test data but found almost nobody in this broadcast - a **domain shift**, so it was replaced.)
- **Tracking. BoT-SORT** links the boxes over time, using the calibration's camera motion. A **registry** (`identity/registry.py`) re-links a player whose track broke, by where they stand **on the pitch in metres** - which camera pans and zooms do not change. The earlier prototype produced 534 identities for about 13 people; CrickSense gives **51 identities for 51 player appearances** on the development clip.
- **Feet on the pitch.** The bottom-centre of each box (the feet) is converted to pitch metres with H - but only when the bottom of the box is not cut off by the edge of the frame or the score bar.
- **Roles** (`identity/role_assigner.py`). Each player builds up evidence for each role from where their feet are: the **striker** near the striker's popping crease, the **keeper** behind the striker's stumps, the **non-striker** beside the bowler's crease off the middle line, the **bowler** at the bowler's crease and **moving**, the **umpire** a few metres behind the bowler's stumps and **standing still**. Evidence decays over about 10 frames. Every frame, the unique roles are assigned jointly with the **Hungarian algorithm**; everyone else is a fielder.
- **Names.** The striker, non-striker and bowler get their names from the score bar; the keeper from the team sheet. Each name goes to the player who held that role most often in the shot.

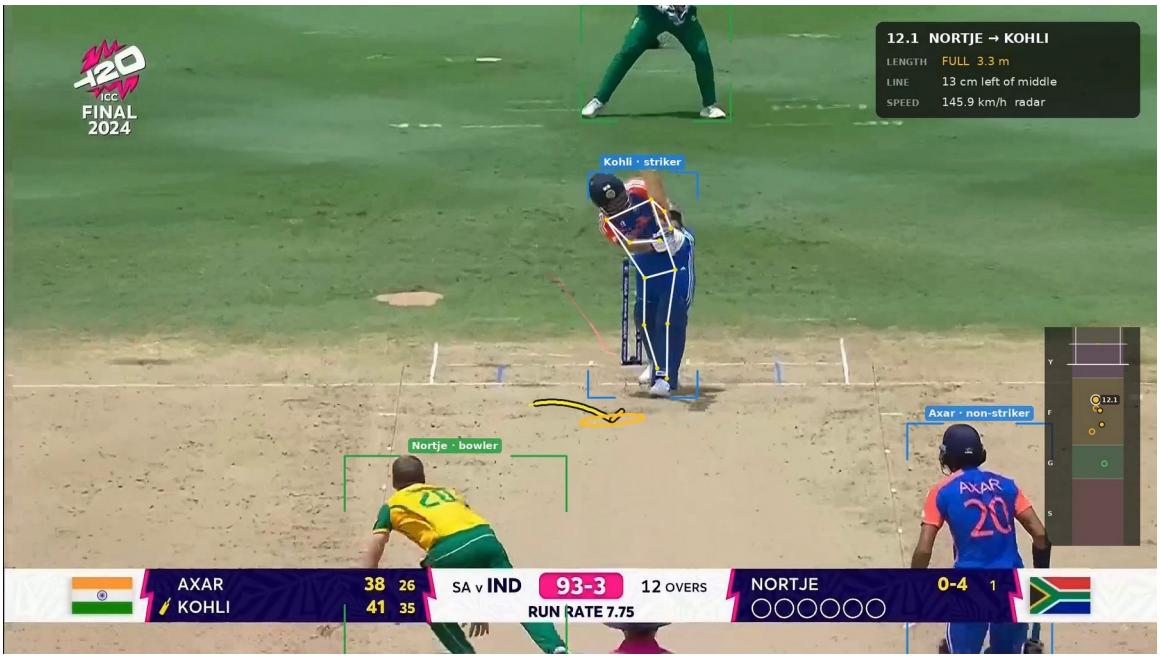


Figure 7.1: Players named by role on the overlay (ball 12.1, Nortje to Kohli), with the striker's pose, the ball's path and the delivery card

CHAPTER 8

Step 4: The Ball

The ball pass (`passes/ball.py`, `ball/delivery.py`, `ball/bounce.py`) finds each delivery and measures where it bounced and how fast it was.

8.1 Finding the Delivery

- **Candidates.** Both ball models run on every calibrated frame at 1920 pixels; any box above a low confidence (0.1) is a candidate. The original model is better at **finding** deliveries (10 of 10); the fine-tuned model is better at **tracking** them (false balls on crease marks fell from 131 to 20).
- **Ground points.** Each candidate is placed where its **sight line** - the line from the camera through that pixel - meets the ground. This removes the camera's pans and zooms.
- **Static candidates removed.** Something that stays at the same ground spot for 5 or more frames is a pitch marking or stump top, not a ball.
- **Release.** A delivery starts where a candidate's sight line passes through the **release zone**: hand height (about 2.1 m) just past the bowler's popping crease.
- **The track.** From the release, one candidate per frame is followed along the delivery's line, under physical limits on how far the ball can move between frames.
- Two deliveries cannot be less than 10 seconds apart; a second "delivery" 2 s after a real one is the ball in play, seen in a wide view, and is dropped.

8.2 The Bounce: a Clever Trick

While the ball is in the air, its ground point (where the sight line meets the ground) lies **ahead** of the ball, by about height × camera distance / camera height. As the ball comes down, the ground point moves back towards the ball and **meets it at the bounce** - the only moment the ground point really is the ball - then jumps ahead again as the ball rises. So the ground point's path turns at the bounce. CrickSense finds the apex of that turn where a line through the falling side meets one through the rising side. If the ball is hit or hidden right after it pitches, the last point before stands in, and the bounce is marked **inferred** (drawn as a ring rather than a dot).

The bounce gives **length** (its x, in metres from the striker's stumps) and **line** (its y, from the middle-stump line). A bounce checked by hand measured 4.77 m against 4.8 m.



Figure 8.1: Every bounce found in the five analysed overs of both matches; rings mark inferred bounces

8.3 Speed

- **The broadcaster's radar first.** When the score bar shows a speed, that is used (17 of 21 balls of the answer-key clips).
- **Otherwise our estimate, from the time of flight.** The ball's flight from release to bounce is modelled with gravity and air drag (`ball/physics.py`), anchored at the two points the camera measures well: the release (on the release sight line at hand height) and the bounce (on the ground, where the calibration is accurate). Roughly, v ≈ distance / (t_bounce − t_release). The median error against radar was **5.7%** on the final.
- **Why not from the ball's motion in the picture?** The earlier prototype did that and showed speeds like 409 km/h. Measured, pixel-displacement speed gave 1-72 km/h against radar readings of 81-146 km/h, because an airborne ball is not on the ground plane and, seen from behind the bowler, it hardly moves across the picture.
- **Pace is the weak case.** A fast ball reaches the bounce about 8 frames after release, so one frame of error is about 13% of the speed, and the first frames of a fast ball are often missed. Estimates above **161.3 km/h** - faster than the fastest ball ever recorded - are therefore withheld.

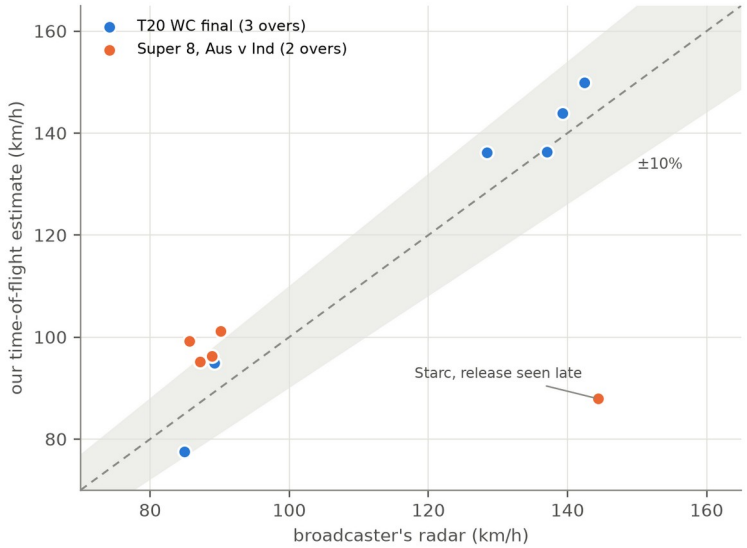


Figure 8.2: Our time-of-flight speed estimate against the broadcaster's radar, for the 11 deliveries with both; the band is ±10%

CHAPTER 9

Step 5: Events and the Score Engine

The fuse pass (`passes/fuse.py`, `fuse/outcome.py`, `fuse/score_engine.py`) turns everything measured so far into **events** and a **match state**.

9.1 What Each Ball Produced

For each delivery, the trusted score-bar state **before** it (the last before its release) and **after** it (the last before the next delivery's release) are compared. The differences give the runs, the batter's own runs (runs off the bat), extras, wickets, whether it was a wide or no-ball, and who was on strike. Runs equal to 4 or 6 off the bat give a **boundary**. Bat contact comes from a bat detector: a bat box touching a ball detection near the striker just after the bounce.

9.2 The Score Engine

The score engine is a **state machine**: the match state (score, wickets, legal balls, who is on strike, each batter's runs and balls, each bowler's figures) is updated from each ball's outcome by the Laws of Cricket:

- strike changes on odd runs and at the end of an over;
- wides do not count as balls faced; byes and leg-byes are not the bowler's runs;
- a new over needs a new bowler; a wicket brings in a new batter.

After every ball the engine's state is **checked against the score bar**, check by check: score, wickets, overs, the striker as predicted, each batter's runs and balls, the bowler's figures. The bar is evidence, not the authority: where the engine has no evidence for a ball (a delivery the ball pass missed), it re-syncs from the bar **and records that it did** - it never silently copies the bar. On five overs of two matches, **230 of 230 checks agreed**.

> **An honest caveat**
> The score and overs checks are consistency checks, because the outcomes themselves come from the same score bar. The checks that test the engine independently are strike rotation, balls faced and the bowler's figures.

CHAPTER 10

Step 6: Batter Pose

The pose pass (`passes/pose.py`, `pose/batter.py`) runs YOLO11x-pose on crops around the striker:

- **Footwork.** The feet on the calibrated pitch at the shot, compared with the stance before release: front foot forward → "front foot"; a foot back towards the stumps → "back foot"; otherwise "in the crease". A foot is trusted only when its ankle is clearly visible and below its knee - in close-ups the model can put "ankles" on the shins.
- **Swing.** The wrists' speed relative to the hips, so camera motion cancels out: fast → the ball was played; slow → left alone. Result: "played" on all 8 balls with runs off the bat and "left" on the one ball not hit.
- **Batting hand.** Right- or left-handed, from where the feet stand at the stance: right on 10 of 10 balls.

Two pose ideas were measured and dropped: **knee on the ground** (to spot sweeps), because from in front a bent knee over a planted foot looks the same; and **front-foot no-balls**, because one pixel covers 3.5 cm at the bowler's crease and the margins are 1-5 cm.

CHAPTER 11

Steps 7-9: Commentary, Voice and the Overlay

11.1 Commentary That Cannot Make Things Up

For each ball a **fact sheet** is built from the passes: over, bowler, striker, length, line, speed and where the speed came from, runs, boundary, wicket, the score, milestones. The LLM (Gemini's free tier, or any OpenAI-compatible service) is given only the fact sheets and asked for one line per ball, plus a line after it when there is more to say. Every line is then **checked** (`commentary/llm.py`):

- every **number** in the line must be a number in that ball's facts - no invented speeds or scores;
- every **player named** must be in that ball's facts;
- no boundary or wicket the facts do not have;
- the line must fit the time before the next ball (a word budget at about 2.7 words per second).

A line that fails is replaced by a **template** line built from the same facts, and the reason is recorded. Answers are cached, so re-running costs no new API calls.

outputs/replay-super-8s_i1_o3/commentary.json - Starc to Rohit, 2.1


```
{
  "engine": "gemini:gemini-3.5-flash-lite",
  "facts": {
    "over": "2.1",
    "bowler": "Starc",
    "striker": "Rohit",
    "length_class": "full",
    "speed_kph": 138.3,
    "speed_source": "radar",
    "runs": 6,
    "boundary": "SIX",
    "score": "12-1"
  },
  "lines": [
    {
      "kind": "ball",
      "source": "llm",
      "text": "Starc bowls a full delivery at 138.3 kph, Rohit plays it from the crease for SIX!"
    },
    {
      "kind": "after",
      "source": "llm",
      "text": "India move to 12-1."
    }
  ]
}
```


Figure 11.1: commentary.json: the facts of Starc's first ball to Rohit and the line the model wrote from them

Shot type was tried with vision-language models and switched off: asked to name a shot they called almost everything a "sweep"; asked only to describe posture, with the shot named by textbook rules, they were consistent on one match but on the second called three different swings "cover drive". So commentary describes only what was measured.

11.2 Voice

Each line is spoken by **Kokoro** with a pace that suits the moment - brisk as the ball is played, excited for a boundary or wicket, calm for the score - and placed at its time on a track the length of the clip; a line that runs long pushes the next one later, never on top of it (`commentary/tts.py`, `passes/voice.py`). Optional engines: Gemini TTS, and F5-TTS, which can speak in the voice of a short recording - used only with the speaker's consent.

11.3 The Overlay and the Crowd Sound

The renderer (`render/overlay.py`) draws on every frame: corner brackets around each player with their name and role, the ball's path held on the pitch while the camera moves (one homography carries any pitch point between frames), the bounce, a **delivery card** (over, bowler → striker, length, line, speed and its source, outcome), banners for fours, sixes and wickets, a pitch-map inset, and the spoken line as a caption. Under the commentary, the stadium sound plays with the **broadcast's own commentators removed** by two passes of Demucs: between our lines, voice was heard for 213 of 290 s in the original sound and 0 s after.

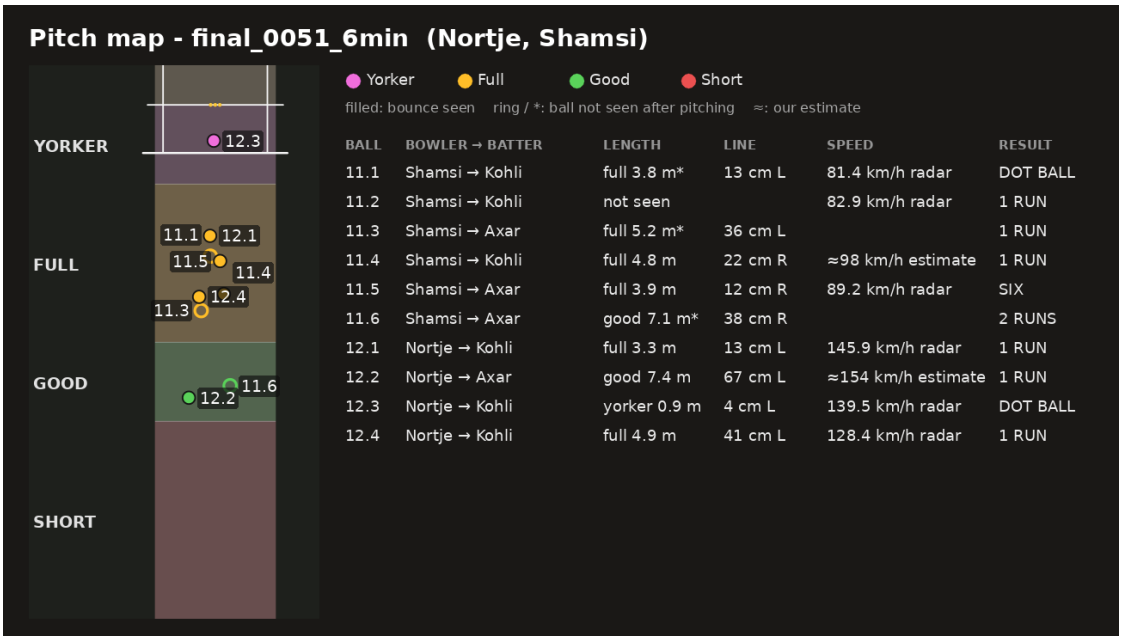


Figure 11.2: The pitch map of the 6-minute clip: each ball's bounce, length zone, line, speed and its source, and outcome

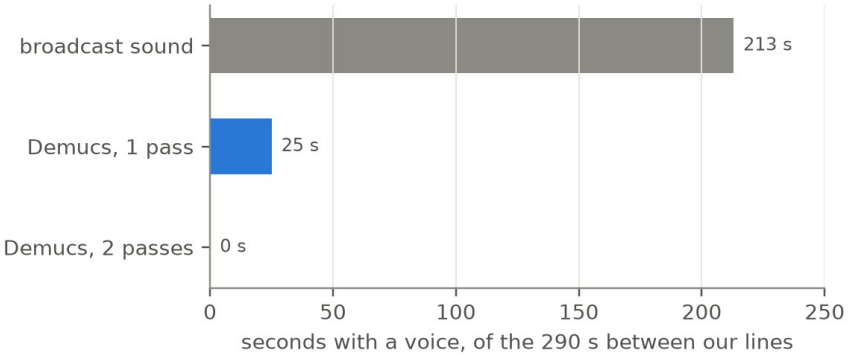


Figure 11.3: Voice heard between our commentary lines: the broadcast sound, and after one and two passes of Demucs

CHAPTER 12

Live Mode

Live mode runs CrickSense on a video **as it plays** - a file at its own speed, or a stream - on one laptop GPU (`live/runner.py`, `live/plus.py`; commands `make live` and `make live-plus`).

12.1 Workers That Never Fall Behind

A **reader** takes frames at the video's own pace and offers each to independent **workers**. A worker that is still busy is given only the **newest** frame, never a backlog, so the system degrades gracefully instead of drifting behind the video:

- the **score worker** reads the bar every 0.5 s and announces each ball's outcome as soon as the bar settles, and every new radar speed;
- the **view-and-ball worker** notices when a main-camera shot starts and ends, then finds that shot's delivery, bounce and our speed estimate;
- the **players worker** detects, tracks and names the players on the newest main-camera frame.

The picture is shown **one second behind** the video, so every box is drawn on the frame it was found on rather than lagging behind a moving camera.

12.2 Making It Fast Enough

- **Flow camera.** The camera is followed by optical flow (about 10 ms a frame) with a full reference match every few frames (40-330 ms); players are placed within 0.25 m (median) of the offline calibration.
- **Half precision.** The ball model became 3.3 times faster with identical detections.
- **Early stop.** A shot's ball analysis stops 12 frames after a confirmed bounce; per shot it fell from 25-66 s to 8-15 s, with lengths within 0.1 m of the offline result on 5 of 6 deliveries.
- **Sharing the GPU.** The ball analysis waits while players are on screen (it had slowed the player detector from 35 to 360 ms a frame) and catches up between balls.
- **Re-calibrating from the creases.** A reference frame only matches the broadcast for minutes, not hours - shadows and grass change. Over a whole match, each delivery's wide view is calibrated directly from its painted creases, confirmed by a second agreeing calibration a few frames later, and shared with the other workers.

12.3 The Delivery Card

The card shows the ball under way or just finished: over, bowler → striker, length and line once the bounce is found, the speed, and the outcome. One subtlety came from measurement: the score bar can show a ball's radar speed **up to 4 seconds after its result**, so a radar reading

within 8 seconds of a result belongs to that ball, not the next; a replay can produce a second, unconfirmed bounce, so the card prefers the confirmed, earlier one.

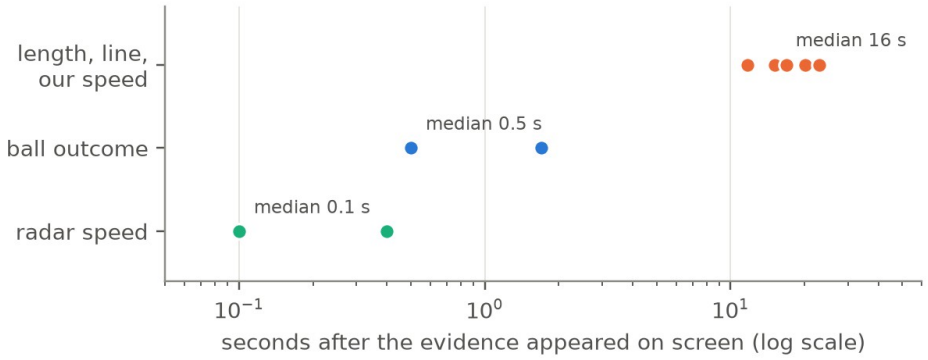


Figure 12.1: Live delay per kind of result on Bumrah's over: how long after the evidence appeared on screen each result was shown

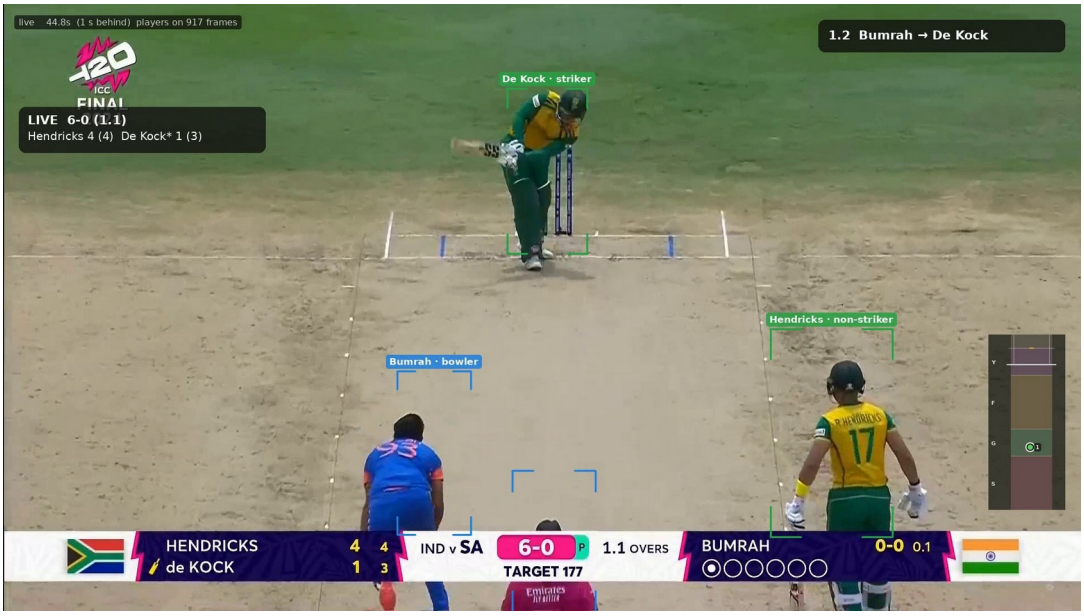


Figure 12.2: The live window on Bumrah's over: striker, non-striker, bowler and umpire named, and the card for ball 1.2

Table 12.1: Live mode results

Measure	Bumrah's over (own calibration)	Whole final, first 7 minutes
Kept pace with the video	327 s for 320 s	425 s for 420 s (on mains power)
Each ball's outcome	6 of 6, 0.5 s after the bar	every ball
Radar speed	about 0.1 s	about 0.1 s
Length, line, our speed	every delivery, 12-23 s after it	most deliveries
Players named	every delivery	about half the deliveries, 1-2 s each

CHAPTER 13

Following One Ball Through the System

This chapter follows one real ball - **ball 1.3 of Bumrah's over** in the final, the wicket of Hendricks - through every step, using the actual output files.

- **Calibration.** The main camera's frames around 94 s are matched to the over's reference views; each frame gets its homography in `camera.json`.
- **Score bar.** Before the ball the bar reads 7-0 after 1.2 overs, Hendricks on strike, Bumrah bowling. After it: 7-1, a new batter, Markram.
- **Players.** On the delivery's frames Bumrah is found running in at the bowler's crease (bowler), Hendricks at the striker's crease (striker), de Kock at the other end (non-striker), the keeper behind the stumps and the umpire still behind the bowler's stumps.
- **Ball.** The release is found at 94.2 s; the ball is followed to its bounce at 94.4 s, **5.35 m** from the striker's stumps (full), 0.23 m to the left of the middle-stump line. The radar on the bar reads **140.5 km/h**.
- **Fuse.** The score changed by 0 runs and 1 wicket on a legal ball: a **WICKET** event with Hendricks as the striker.

outputs/st_bumrah_o2/events.json - the four (1.4) and the wicket (1.3)

```
[
  {
    "frame_id": 5760,
    "t": 230.46,
    "type": "RUNS",
    "payload": {
      "delivery_id": 3,
      "bowler": "BUMRAH",
      "striker": "MARKRAM",
      "over": "1.4",
      "runs": 4,
      "off_bat": 4,
      "extras": 0,
      "kind": "legal",
      "score": "11-1"
    },
    "confidence": 0.9,
    "source": "scoreboard"
  },
  {
    "frame_id": 2412,
    "t": 96.5,
    "type": "WICKET",
    "payload": {
      "delivery_id": 2,
      "bowler": "BUMRAH",
      "striker": "HENDRICKS",
      "over": "1.3",
      "score": "7-1"
    },
    "confidence": 0.9,
    "source": "scoreboard"
  }
]
```

Figure 13.1: events.json: the four of ball 1.4 and the wicket of ball 1.3, each with its time, source and confidence

The score engine then updates the state by the Laws - a wicket, the incoming batter on strike - and checks it against the bar. Every check is true, and a note records that the incoming batter and the strike came from the score bar:

outputs/st_bumrah_o2/match.json - the state after the wicket


```
{
  "over": "1.3",
  "runs": 7,
  "wickets": 1,
  "legal_balls": 9,
  "striker": "MARKRAM",
  "non_striker": "DE KOCK",
  "bowler": "BUMRAH",
  "batters": {
    "HENDRICKS": [4, 5],
    "DE KOCK": [1, 4],
    "MARKRAM": [0, 0]
  },
  "bowlers": {
    "BUMRAH": [0, 3, 1]
  },
  "source": "engine",
  "checks": {
    "same bowler within the over": true,
    "striker as predicted": true,
    "score": true,
    "wickets": true,
    "overs": true,
    "Markram runs/balls": true,
    "De Kock runs/balls": true,
    "on strike": true,
    "Bumrah figures": true
  },
  "notes": ["wicket: incoming batter and strike from the score bar"]
}
```


Figure 13.2: match.json: the state after the wicket, with every check against the score bar

Finally the commentary pass builds the fact sheet (over 1.3, Bumrah to Hendricks, full, 140.5 km/h radar, wicket, 7-1), the voice pass speaks the checked line, and the overlay shows the card, a WICKET banner and the caption. The ball's own record in `ball.json` (here for ball 1.1) looks like this:

outputs/st_bumrah_o2/ball.json - delivery 1.1


```
{
  "delivery_id": 0,
  "release_frame": 298,
  "release_t": 11.9,
  "release_point": [18.66, -0.8, 2.0],
  "detections": 12,
  "bounce_t": 12.2,
  "bounce_xy": [7.4, -0.08],
  "bounce_confirmed": true,
  "length_m": 7.4,
  "line_m": -0.08,
  "length_class": "good",
  "radar_speed_kph": 139.3,
  "speed_estimate_kph": 143.88,
  "track_model": "ball_ft.pt"
}
```


Figure 13.3: ball.json: one delivery's release, bounce, length, line and both speeds

CHAPTER 14

Testing and Measurement

14.1 How Results Were Measured

- **Held-out clips.** Three clips form the **answer key**: India's overs 11.1-12.4 (development), and two clips never seen during development - Kuldeep's spin over and Bumrah's pace over. Every ball's over, score, striker, bowler and radar speed was checked by hand against the broadcast (`configs/golden.yaml`).
- **Regression check.** `make check` re-runs those clips after any change and compares them ball by ball; it prints PASS or FAIL and exactly which ball changed.
- **Benchmark.** `make benchmark` runs 20 random clips of the final with no clicks, to see how the whole chain does on footage it has never seen.
- **Unit tests.** 200 automated tests (`make test`) check each piece, from homography maths to the live card's timing rules.
- **References.** Where the broadcaster's own data is the reference (radar, the score bar), that is said.

14.2 All Results

Table 14.1: Measured results

Area	Measure	Result
Calibration	corners vs hand clicks	7 of 8 within 3.3 px
Calibration	20 random clips	14 calibrated, none wrongly
Score bar	two whole matches	4 of 4 final scores exact; 117-121 of 121 ball counts per innings
Score bar	digit templates	250 of 250 (leave-one-out)
Ball	answer-key clips	21 of 21 deliveries; 18 bounces
Ball	a bounce checked by hand	4.77 m vs 4.8 m
Speed	estimate vs radar	5.7% median (final); 8.3% (both matches)
Outcomes	answer-key clips	21 of 21 right
Score engine	five overs, two matches	230 of 230 checks
Players	development clip	51 identities for 51 appearances
Pose	swing / batting hand	9 of 9 / 10 of 10
Voice	TV commentators removed	213 s → 0 s of voice between our lines
Live	outcomes / radar	0.5 s / 0.1 s after the bar

14.3 Processing Time

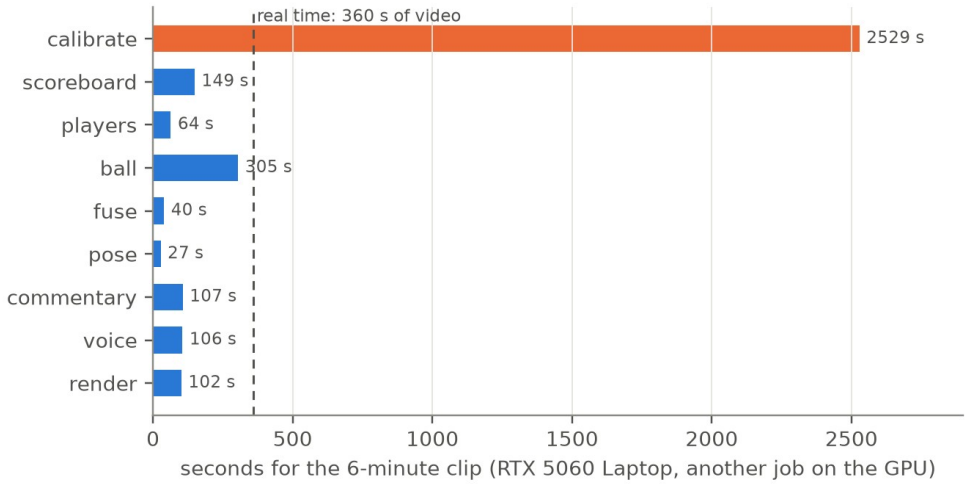


Figure 14.1: Time of each offline pass on the 6-minute clip (RTX 5060 Laptop GPU). Every pass but calibration runs faster than real time

CHAPTER 15

What Did Not Work, and Why

Knowing what failed is part of understanding the project. Each of these was measured before it was dropped.

Table 15.1: Approaches measured and dropped

Approach	What was measured	What was done instead
Speed from the ball's pixel motion	1-72 km/h against radar 81-146	radar from the bar; time-of-flight estimate
Runs from the batters running	the pitch is on screen at most 1.8 s after the shot on 14 of 15 balls; a run takes 3-3.5 s	runs from the score bar
Wides from the ball's position	legal balls measured 1.3-2.75 m wide	wides from the score bar
Front-foot no-ball	3.5 cm per pixel at the crease; margins 1-5 cm	no-balls from the score bar
A fielder map	the main view shows 5 people at the delivery	not attempted
Retrained 5-class player model	0.895 mAP50 on its own data; almost no detections here	COCO model + roles from geometry
Tesseract on score digits	"11-1" read as "1-1", "11" or "-1"	font templates (250/250)
Shot names from vision-language models	"sweep" for almost every shot; unstable between runs	switched off
Knee-down from pose	true on an upright batter	dropped
Half-resolution camera tracking	1.9× faster, corner error 1.8 → 3.1 px	full resolution kept offline
Release always from the bowler's arm	speed error 6% → 22%	kept only as a fallback

CHAPTER 16

The Code, Files and Configuration

16.1 Where Everything Lives

```
CrickSense/
  main.py                  the one command: python main.py <video>
  Makefile                 one 'make' command per task (make help lists them)
  src/cricsense/
    passes/                one file per pass: calibrate, scoreboard, players, ball,
                           fuse, pose, commentary, voice, render, publish, run
    calibration/           auto (creases), homography, camera_motion, camera_pose
    identity/              scoreboard_ocr, digits, registry, role_assigner,
match_sheet
    track/                 player_tracker (BoT-SORT with calibrated camera motion)
    ball/                  candidates, delivery, bounce, physics
    fuse/                  outcome, score_engine, contact, overs
    pose/                  batter, bowler
    commentary/            facts, llm, templates, tts, vlm
    render/                overlay, pitch_map, wagon_wheel, draw, crowd (Demucs)
    live/                  runner (live mode), plus (live-plus)
    schema/                the JSON file formats (Pydantic models)
    store/, api/           PostgreSQL, object storage, FastAPI
  configs/                 every threshold, the layout, team sheets, calibrations
  scripts/                 tools: make_clip, check, benchmark, label_ball, doctor ...
  tests/                   200 automated tests
  docs/                    REPORT, RUNBOOK, decisions log, the lab report and slides
  data/  outputs/  models/ clips, results and model weights (not in git)
```

16.2 Configuration Files

Table 16.1: The main configuration files

File	What it holds
configs/default.yaml	paths, the GPU device, camera-tracking settings, the masked screen areas
configs/scoreboard.yaml	the score-bar layout: every field's rectangle, colours, the radar line
configs/matches/<match>.yaml	the team sheet: both squads as the bar spells them, keepers
configs/calibration/<clip>.yaml	a clip's reference views and their homographies
configs/ball.yaml, players.yaml	detector weights, image sizes, length zones, tracker choice
configs/commentary.yaml	the LLM provider and model, the TTS voice, commentator removal
configs/golden.yaml	the hand-verified answer key used by make check
configs/clips.yaml	where each clip was cut from, so its sound can be found
.env (not in git)	API keys, e.g. GEMINI_API_KEY - never committed

16.3 The Output Folder of a Clip

Table 16.2: Files in outputs/<clip>/

File	What it is
overlay.mp4	the annotated video with spoken commentary
calibration_check.png	the creases drawn through the calibration - check it by eye
pitch_map.png, wagon_wheel.png	where each ball bounced; where the runs went
camera.json	the homography of every frame
scoreboard.json	every score-bar reading
players.json, pose.json	every person per frame with role and name; the striker's body
ball.json	each delivery: release, track, bounce, length, line, speeds
events.json, match.json	the events; the match state after every ball, with its checks
commentary.json, commentary.wav	the lines with their facts and checks; the voice track

CHAPTER 17

Running the Project

17.1 Setting Up

```
pip install -r requirements.txt     # in the conda environment 'ball'
make models                          # downloads every model (~560 MB)
make doctor                          # says READY, or what is missing and how to fix
it
make test                            # the 200 tests (~30 s)
```

For commentary written by the language model, create a free key at aistudio.google.com/apikey and add the line `GEMINI_API_KEY=...` to the `.env` file; `make llm-check` tests it. Without a key, template commentary is used and everything else works.

17.2 Everyday Commands

Table 17.1: Commands

Command	What it does
python main.py data/clip.mp4	analyse a clip in full
python main.py match.mp4	a whole match: final scores and every over listed
python main.py match.mp4 --overs 1:12 2:2	analyse chosen overs (or big, or all)
make clip START=6840 END=7160 CLIP=name	cut a clip from the match
make run CLIP=name	every pass on a clip, with voice
make check	re-run the answer-key clips and compare ball by ball
make demo	join clips into the demo video
make live-plus CLIP=name	live window: names, card, speed
make live-plus VIDEO="match.mp4" CAL=configs/calibration/x.yaml	live on a whole match
make serve	the HTTP API at localhost:8000/docs

17.3 A New Match

- Put the video in `Cricket match/`.
- Copy a team sheet in `configs/matches/`, named like the video, and set both squads exactly as the score bar spells them and the keepers.
- Cut a 5-minute clip of play (`make clip MATCH=... START=... END=... CLIP=...`) and calibrate it (`python -m cricsense calibrate --video data/<clip>.mp4 --auto`); check `calibration_check.png` by eye.
- Analyse overs with `main.py`, or run live with `make live-plus VIDEO=... CAL=...`.
- A broadcaster with a different score bar needs a new layout file and new digit templates first (`make digits`); see `docs/RUNBOOK.md` section 7.

CHAPTER 18

How the Project Developed

CrickSense started from the team's internship prototype (January 2026). Its claims were **checked first**, and several did not hold: speeds above 400 km/h shown on screen came from pixel displacement; the player model, retrained, found almost nobody in this broadcast; the tracker produced 534 identities for about 13 people; the score-bar reader could not read a name. CrickSense was rebuilt around measurement, with the earlier problems recorded in the code where they were fixed.

Table 18.1: Timeline (from the project's commit history)

Dates (2026)	What was built
27-29 September	package skeleton, video reading, calibration with camera tracking; players with stable IDs, roles and names; the ball pass
30 September - 1 October	fuse and score engine, overlay, commentary, voice; automatic calibration from the creases; whole-match score bar; storage, API and live mode
2-14 October	pace (Bumrah's over): several references per camera, score-box font templates; the answer key and regression check; the runbook
15-20 October	live mode re-measured; voice options; per-match team sheets; the second match (Super 8); the one-command pipeline
21-28 October	commentary on Gemini's free tier; TV commentators removed with Demucs; live-plus with named players and the delivery card
29 October	live naming over a whole match by re-calibrating from the creases; the lab report and presentation

CHAPTER 19

Limitations and Future Work

19.1 Limitations

- **Pace speed at 25 fps.** One frame is about 13% of a fast ball's flight to the bounce; estimates for pace are the least accurate.
- **Missed deliveries.** A delivery is missed when the ball is not detected near the release (3 of 14 calibrated benchmark clips).
- **Live naming over a whole match** works on about half the deliveries, for 1-2 s each, because each delivery must be re-calibrated within the first second of its shot.
- **One broadcaster's graphics.** Another score bar needs its own layout and digit templates; the ball model may need a short labelling round.
- **Not visible, not measured.** Runs by running, wides, no-balls and the field setting come from the score bar or are not attempted.

19.2 Future Work

- 50 fps footage and labelled frames of the ball's first flight, for pace speed and missed deliveries.
- Faster crease calibration, so that players are named live through a whole match.
- A labelled answer key for shot types, so that a vision-language model can be scored before it is used.
- Score-bar layouts for other broadcasters; multi-camera input for 3D ball trajectories where available.

CHAPTER 20
Glossary

Table 20.1: Terms used in this guide

Term	Meaning
Answer key	every ball of three clips checked by hand; used to test the system
BoT-SORT	a multi-object tracker with camera-motion compensation
Bounce (confirmed / inferred)	where the ball pitched; confirmed when seen after the turn, inferred otherwise
Calibration	knowing which pixel is which point on the pitch, through a homography
Crease	a painted white line on the pitch, at positions fixed by the Laws
Delivery card	the box on screen with a ball's over, bowler, striker, length, line, speed and result
Demucs	a model that separates voices from the rest of a soundtrack
Domain shift	a model trained on one kind of image failing on another
FP16	16-bit numbers on the GPU; faster inference
Hallucination	a language model stating something not in its input
Homography	a 3 × 3 matrix mapping points of a plane between two views
Hungarian algorithm	finds the best one-to-one assignment, e.g. players to roles
Inlier	a feature match that agrees with the fitted model in RANSAC
JSON	a text format for structured data; every pass writes one
Kalman filter	estimates position and velocity from noisy measurements over time
Length / line	the bounce's distance from the striker's stumps / from the middle-stump line
LLM	large language model, used here to write commentary lines
Optical flow	how image points move between two frames
ORB	a fast image-feature detector and descriptor
Pass	one step of the offline pipeline, reading and writing files
RANSAC	fits a model robustly when some data points are wrong
Score engine	the state machine that keeps the score by the Laws
Template matching	recognising a shape by comparing it with stored pictures
Time of flight	the time from release to bounce, used for our speed estimate
YOLO	a fast one-pass object detector

References

[1] G. Thomas, R. Gade, T. B. Moeslund, P. Carr and A. Hilton, "Computer vision for sports: Current applications and research topics," Computer Vision and Image Understanding, vol. 159, pp. 3-18, 2017.

[2] N. Owens, C. Harris and C. Stennett, "Hawk-eye tennis system," in Proc. Int. Conf. Visual Information Engineering, 2003, pp. 182-185.

[3] K. Abbas et al., "Deep-learning-based computer vision approach for the segmentation of ball deliveries and tracking in cricket," arXiv:2211.12009, 2022.

[4] Y.-C. Huang et al., "TrackNet: A deep learning network for tracking high-speed and tiny objects in sports applications," in Proc. IEEE AVSS, 2019.

[5] R. Voeikov, N. Falaleev and R. Baikulov, "TTNet: Real-time temporal and spatial video analysis of table tennis," in Proc. IEEE/CVF CVPR Workshops, 2020.

[6] R. Hartley and A. Zisserman, Multiple View Geometry in Computer Vision, 2nd ed. Cambridge University Press, 2004.

[7] N. Homayounfar, S. Fidler and R. Urtasun, "Sports field localization via deep structured models," in Proc. IEEE CVPR, 2017.

[8] J. Chen and J. J. Little, "Sports camera calibration via synthetic data," in Proc. IEEE/CVF CVPR Workshops, 2019.

[9] Marylebone Cricket Club, The Laws of Cricket, 2017 Code (3rd ed., 2022).

[10] J. Redmon, S. Divvala, R. Girshick and A. Farhadi, "You only look once: Unified, real-time object detection," in Proc. IEEE CVPR, 2016.

[11] G. Jocher, A. Chaurasia and J. Qiu, "Ultralytics YOLO (YOLOv8, YOLO11)," software, 2023-2024. https://github.com/ultralytics/ultralytics

[12] Y. Zhang et al., "ByteTrack: Multi-object tracking by associating every detection box," in Proc. ECCV, 2022.

[13] N. Aharon, R. Orfaig and B.-Z. Bobrovsky, "BoT-SORT: Robust associations multi-pedestrian tracking," arXiv:2206.14651, 2022.

[14] E. Rublee, V. Rabaud, K. Konolige and G. Bradski, "ORB: An efficient alternative to SIFT or SURF," in Proc. IEEE ICCV, 2011.

[15] M. A. Fischler and R. C. Bolles, "Random sample consensus," Communications of the ACM, vol. 24, no. 6, pp. 381-395, 1981.

[16] B. D. Lucas and T. Kanade, "An iterative image registration technique with an application to stereo vision," in Proc. IJCAI, 1981.

[17] J. Shi and C. Tomasi, "Good features to track," in Proc. IEEE CVPR, 1994.

[18] R. Smith, "An overview of the Tesseract OCR engine," in Proc. ICDAR, 2007.

[19] Z. Ji et al., "Survey of hallucination in natural language generation," ACM Computing Surveys, vol. 55, no. 12, 2023.

[20] S. Rouard, F. Massa and A. Défossez, "Hybrid transformers for music source separation," in Proc. IEEE ICASSP, 2023.

[21] A. Savitzky and M. J. E. Golay, "Smoothing and differentiation of data by simplified least squares procedures," Analytical Chemistry, vol. 36, no. 8, 1964.

[22] hexgrad, "Kokoro-82M: an open-weight text-to-speech model," model card, 2025. https://huggingface.co/hexgrad/Kokoro-82M

[23] T.-Y. Lin et al., "Microsoft COCO: Common objects in context," in Proc. ECCV, 2014.